KANERVA: Token-Budgeted, Validity-Aware Memory for Coding Agents, Without an LLM in the Loop

Artem Katolikov*

October 2026

Abstract

Coding agents forget everything between sessions, and the memory systems built for them mostly select context by similarity top-k, ignore what items cost in tokens, and have no model of whether a stored fact is still true. We cast agent memory as online budgeted selection, show that the surveyed systems are special cases, and derive where each is worst-case suboptimal. We then present KANERVA, a local memory layer with no LLM in the loop. It separates an item's *validity* from its *usefulness*, selects under a hard token budget with a monotone submodular objective and a per-query optimality certificate, estimates validity from the store's own version history, and learns a linear policy from execution feedback. We prove a $\frac{1}{2}(1 - \frac{1}{e})$ selection guarantee, a dynamic regret bound in which the price of expiring items is exactly the accumulated validity error, a linear lower bound for validity-blind learners, a rate–distortion bound for reversible consolidation, and a stationary store-size result. On retrieval-level metrics, without an LLM judge, KANERVA raises recall at 1024 tokens from 0.761 to 0.860 on LongMemEval-S and from 0.682 to 0.722 on LoCoMo against hybrid retrieval. On RepoMem, a new benchmark built from real repository histories, it cuts stale-fact errors on changed facts from 0.700 to 0.030 without any structured keys. It loses to hybrid retrieval at large budgets on LoCoMo and by a small margin in recall on RepoMem, and we report those cases in full.

1 Introduction

A coding agent starts each session knowing nothing it learned in the last one. Native memory is a set of files loaded whole: Claude Code loads its `CLAUDE.md` files and the first 200 lines or 25 KB of a memory index, with no ranking by the task at hand [4]. Memory products add retrieval, but they select by similarity top-k or leave the choice to the agent, and they spend LLM calls on every write. The published evidence does not show that this is good enough. On LoCoMo, Mem0's own table has a full-context baseline ahead of every memory system tested [12]. Retrieval method moves accuracy far more than write strategy, and raw chunks with no LLM calls match fact extraction [68]. On coding tasks, unfiltered experience gives little or negative benefit [74], and eleven of twelve pairings of solvers with memory systems fail to beat memory-off [16].

These findings point at the read path, and at three defects in it. Selection ignores what an item costs in tokens. It ignores what the other selected items already say. And it ignores whether an item is still true, which is the common failure in a codebase, where a function signature read last month may have changed since.

This paper treats memory as an online budgeted selection problem and makes four contributions.

1. A framework in which the surveyed systems are special cases, with elementary worst-case arguments for what each one gives up (Section 3).

2. A method that separates *validity* from *usefulness*: budgeted monotone submodular selection with a per-query optimality certificate, a validity model estimated from the store's own version history, and a linear policy learned from execution feedback (Section 4). It runs locally and needs no LLM.

3. A regret decomposition for items that expire (Theorem 2). Stationary linear submodular bandit results exist [57, 67, 69]; they exclude this case. Here the cost of non-stationarity is exactly the accumulated validity error, while a validity-blind learner has linear regret (Proposition 8). The other theory results are a rate–distortion bound for reversible consolidation and a stability result for store size.

4. RepoMem, a coding-memory benchmark built from the real histories of eight Python repositories, in which facts change for real and every label comes from the code by static analysis. We evaluate on it, on LongMemEval-S and on LoCoMo against nine reading baselines and Mem0's verbatim mode, with ablations, confidence intervals and the cases where the method loses.

*Drafted with Claude Code. Code, data pointers and every reported number are at the project repository; see the Reproducibility section.

Scope of the evidence. All results here are *retrieval-level*: evidence recall under a token budget, tokens injected, staleness errors, latency and store size. No LLM was available for these experiments, so we report no answer accuracy and no end-to-end task success. Recall under a budget is a necessary condition for those outcomes, not a measurement of them. A controlled study found that context strategy moved correctness on coding tasks by no more than 10 to 15 points [28], so large end-to-end gains should not be assumed.

2 Related work

Memory systems. Memory layers include OS-style paging by the agent [43], LLM-curated fact stores [12], temporal knowledge graphs with bi-temporal edge invalidation [49], linked notes [66], graph retrieval [15, 20, 21], and recursive summary trees [51]. Generative Agents scores memories by recency, importance and relevance [45]. Experience memories such as Reflexion, Voyager, ACE and ReasoningBank keep lessons or skills [42, 52, 62, 70]. All of these use a write path with LLM calls, a read path with a fixed k or an agent decision, or both. Of these systems, Zep/Graphiti is the one with a mechanism for validity. It is binary and depends on the extractor producing the invalidating edge.

Budgeted selection. Token-budgeted submodular selection for context is an active preprint line. PACMS applies facility location under a knapsack as a pluggable engine and lists learned relevance as future work [19]. AdaGReS scores relevance against redundancy [46], and PC-SubMax compresses prompts with a regularized objective [73]. The theory rests on Nemhauser et al. [41], on budgeted maximum coverage [29], on partial enumeration [17, 56], and on cost-benefit greedy with lazy evaluation [30, 34, 39]. MMR [8] and DPPs [31] target diversity without a budget in tokens.

Learning. Combinatorial and linear semi-bandits give regret with approximate oracles [9, 10, 33, 63], and linear submodular bandits extend this to diversified retrieval and to knapsacks [57, 67, 69]. Our analysis uses the confidence sets of OFUL [1]. In agent memory, DeMem proves regret bounds for a K-state memory partition [75] and MemRL shows Q-value convergence [71]; RSCB-MC and MERA use bandits to choose agent actions in coding [24, 25]. None of these treats token-budgeted selection of items whose validity changes. On reward signals, self-scored episodes inflate wrong answers [5], so we use execution signals.

Validity and forgetting. ACT-R base-level activation models the power law of forgetting [2, 3], and it has been applied to LLM memory [22]. Staleness is a measured failure: with a budget of two records, models acted on stale records in most episodes of one controlled study [40], and conflict benchmarks exist for chat [59]. Consolidation follows complementary learning systems [32, 38]. Rate–distortion views of compaction appear in [13, 53, 75]. Colaco and Lahjouji argue that repeated irreversible summarization degrades end-task error faster than retrieval-backed memory, which agrees with the context collapse reported for ACE [70], and motivates keeping sources.

Compression and parametric memory. Text compression such as LLMLingua [26, 27, 44] works with closed APIs and is complementary to selection. Gist tokens, AutoCompressors, ICAE, and KV-cache eviction [11, 18, 36, 47, 65, 72] and test-time weight updates [6, 7, 54] need access to model internals that closed APIs do not expose.

Benchmarks. LoCoMo [37], LongMemEval [64], MemoryAgentBench [23] and BEAM [60] cover conversational memory. SWE-ContextBench [74] and VibeMemBench [16] cover coding experience, MemoryCode covers multi-session coding instructions on synthetic data [48], and ContextBench covers retrieval within a task [35]. None of these, as far as we found, tests repository facts that change between sessions over real history.

3 A unified framework

Memory. At time t the store is a finite set M_t. An item is a tuple $m = (\text{text}, e, \tau^{\text{c}}, \tau^{\text{v}}, \pi, c, k, P)$: content, unit embedding, creation time, last time the item was confirmed, provenance, token cost $c(m) \in \mathbb{N}_{>0}$, an optional *slot key* k that identifies the fact the item states, and parent links P to the items it was consolidated from. Items sharing a key form a version chain.

Validity and usefulness. Fix a step context (query) q. Let $\rho_t(m) = \Pr[m \text{ still holds at } t]$ and $\mu_q(m) = \Pr[m \text{ helps with } q \mid m \text{ holds}]$. The *effective weight* of an item is $w_q(m) = \rho_t(m)\,\mu_q(m)$. Separating the two matters: ρ moves with time and changes in the world, μ with the query. Section 5 shows that a learner which conflates them has linear regret.

Selection. Injecting $S \subseteq M_t$ has utility $U(S \mid q)$, and the context budget is a hard constraint:

$$\max_{S \subseteq M_t} \mathbb{E}\left[U(S \mid q_t)\right] \quad \text{s.t.} \quad c(S) := \sum_{m \in S} c(m) \le B. \quad (1)$$

We study the family

$$F(S) = (1-\gamma)\Big[\sum_{f\in\mathcal{F}_q} \omega_f \Big(1-\prod_{m\in S}\big(1-w(m)\,a_{mf}\big)\Big)+\eta \sum_{m\in S} w(m)\Big] + \gamma \sum_{c\in C}\sum_{m\in S} p_c \max\{0, s(c,m)\}. \tag{2}$$

Here $\mathcal{F}_q$ is a set of facets of the query with weights $\omega_f \geq 0$, $\sum_f \omega_f = 1$, and $a_{mf} \in [0,1]$ is the chance that item m answers facet f given that it is valid and useful. The first term is the expected fraction of the need that gets covered when items succeed independently, the second the expected number of useful items injected, and the third a facility-location coverage of the candidate pool C with similarity $s \in [0,1]$ and fixed importance p_c, which removes near-duplicates.

Lemma 1. *For fixed $w \in [0,1]^M$, F is non-negative, monotone and submodular, with $F(\emptyset) = 0$. It is coordinate-wise non-decreasing in w and $|F(S;w) - F(S;w')| \leq L\sum_{m\in S}|w(m) - w'(m)|$ with $L = (1-\gamma)(1+\eta)$.*

The online problem. A memory policy $\Pi = (\mathsf{W}, \mathsf{R}, \mathsf{C}, \mathsf{F})$ decides what to write, what to read, what to consolidate and what to forget. For a sequence of steps it earns

$$J(\Pi) = \mathbb{E}\sum_{t=1}^{T}\Big[U(S_t \mid q_t) - \kappa_{\mathrm{r}}\, c(S_t)\Big] - \kappa_{\mathrm{w}}\,\mathrm{WriteCost}(\Pi) - \kappa_{\mathrm{s}}\sum_t a(M_t), \quad c(S_t) \leq B \;\forall t. \tag{3}$$

Reading costs tokens on every step, writing costs LLM calls or latency, and the store costs retrieval time and noise. Learning, in this paper, is updating Π and M_t; model weights never change.

3.1 Existing methods as special cases

Table 1 places the surveyed systems in (1)–(3). Nearly all read policies reduce to the modular case $\eta = 1, \gamma = 0$, $\mathcal{F}_q = \emptyset$ with a cardinality constraint and a method-specific score. The systems differ mostly in what they store, which is the W component.

Parametric methods (test-time training [54, 55], memory layers [7], Titans [6], gist tokens and KV-cache eviction [36, 47, 72]) keep memory in weights or attention state. They fall outside the framework by construction, and they require write access to weights or to the cache, which a closed API does not expose.

3.2 Where the special cases are suboptimal

The following statements are worst-case and elementary; the point is that each identifies a design defect that a better method must remove. Proofs are in Appendix A. Write OPT for the optimum of (1).

Proposition 1 (Cost blindness)**.** *Any rule that fills the budget in the order of a score independent of c (top-k fill, MMR, reciprocal rank fusion, reranking) has worst-case ratio at most $c_{\min}/B$ against* OPT*, even for additive U.*

Proposition 2 (Redundancy blindness)**.** *Any modular rule has worst-case ratio at most $1/k$ on coverage objectives with k facets when one facet has at least k near-duplicate items.*

Proposition 3 (Recency as proxy)**.** *A relevant item followed by more than $B/c_{\min}$ later items is never injected by recency-only selection, however relevant it is, whereas a rule that scores by relevance injects it whenever it outranks those items.*

Proposition 4 (Time-invariant scoring)**.** *Any selector whose score is invariant under permuting the timestamps of items with equal content features injects an outdated version ahead of the current one with probability at least $1/2$ on version pairs of equal text similarity to the query.*

Proposition 5 (Fixed policies)**.** *For any fixed linear scoring vector θ there is a $\theta^\star$ for which the fixed policy earns regret $\Omega(T)$, while the learner of Theorem 2 has regret $\tilde{O}(\sqrt{T})$ on the same instance.*

Table 1 shows which propositions apply: every k-limited row is cost-blind, every modular row is redundancy-blind, every row with no validity model or a recency proxy falls under Propositions 3–4, and every non-learning row under Proposition 5. The systems with invalidation (Zep/Graphiti) avoid Proposition 4 only for facts where the extractor produced the invalidating edge.

4 Method

KANERVA is the combination of the components below, implemented with SQLite, FTS5 and NumPy. No component calls an LLM. When a host agent is present it can supply structured writes through tools, which is how semantic extraction happens at no added API cost.

4.1 Write path

A write redacts secrets, deduplicates against live items (an identical write only refreshes the confirmation time τ^{v}), and stores the item with its FTS5 entry and embedding. A write with a key closes the version interval of the live item with that key: the old item keeps its text and gets τ^{end} and a pointer to its successor. Nothing is rewritten, so history stays queryable.

4.2 Read path

Algorithm 1 shows retrieval. Candidates are the union of the top-100 lexical and dense hits. Features combine

Table 1: Surveyed methods as instances of the framework. "Score" is the w used by the read policy; k denotes a cardinality limit.

Method	Write W	Score w	Constraint	Validity ρ
Top-k RAG, hybrid, rerank [14, 50]	chunks	BM25/cosine/RRF/CE	k	none
MMR [8]	chunks	greedy rule, λ trade-off	k	none
PACMS, AdaGReS [19, 46]	chunks	facility location; relevance−redundancy	$c(S) \le B$	none
MemGPT/Letta [43]	agent tool calls	LLM decides paging	block sizes	none
Mem0 [12]	LLM add/update/delete	similarity	k	LLM rewr
Zep/Graphiti [49]	LLM graph extraction	hybrid + rerank	k	$\rho \in \{0,1\}$
A-MEM [66]	LLM notes and links	similarity + links	k	LLM note
HippoRAG 2 [21]	OpenIE triples	personalized PageRank	k	none
RAPTOR [51]	recursive summaries	similarity on all levels	k or tokens	none
Generative Agents [45]	observations, reflections	recency+importance+relevance	k	recency pr
ACE, ReasoningBank [42, 70]	LLM-curated lessons	similarity or whole playbook	k	none
Claude Code files [4]	model-decided notes	constant (query-independent)	200 lines/25 KB	none
Full history	everything	constant	window size	none
LLMLingua family [26, 44]	—	—	compresses $c(m)$	none

Algorithm 1 RETRIEVE(q, B, t)

$C \leftarrow$ top-N of FTS5 $\cup$ top-N of dense search
for $m \in C$ **do**
  $x_m \leftarrow$ features of (m, q); $\mu_m \leftarrow \mathrm{clip}_{[0,1]}\big(x_m^\top \hat{\theta} + \beta \|x_m\|_{V^{-1}}\big)$
  $\rho_m \leftarrow \rho_{\text{stale}}$ if m is superseded and q has no historical intent, else Lomax survival (4) at $t - \tau^{\text{v}}(m)$
  $\rho_m \leftarrow \rho_m \prod_{m' \text{ newer}} \big(1 - \pi\, g(s(m, m'))\big)$ ▷ soft supersession
  $w_m \leftarrow \rho_m \mu_m$
end for
$\mathcal{S} \leftarrow \{\text{GREEDY}(F, B, \varrho) : \varrho \in \{1, \frac{1}{2}, 0\}\} \cup \{\arg\max_{c(m) \le B} F(\{m\})\}$
return $\arg\max_{S \in \mathcal{S}} F(S)$ with certificate (5)

Algorithm 2 GREEDY(F, B, ϱ)

$S \leftarrow \emptyset$; $A \leftarrow \{m : c(m) \le B\}$
while $A \neq \emptyset$ **do**
  $m \leftarrow \arg\max_{m \in A} \left[F(S \cup \{m\}) - F(S)\right] / c(m)^{\varrho}$; remove m from A
  if the gain is 0 **then break**
  end if
  if $c(S) + c(m) \le B$ **then** $S \leftarrow S \cup \{m\}$
  end if
end while
return S

rank and score signals, query-term coverage, ACT-R base-level activation [3], and an item's feedback posterior, in $x(m, q) \in \mathbb{R}^{12}$. Facets are the query's content terms with *idf* weights plus one facet for the query as a whole.

Only the density variant $\varrho = 1$ and the singleton carry the guarantee; the other runs can only raise F, because the maximum is taken over all runs.

4.3 Validity

For a class k of facts (for example, function signatures), the unknown change rate has a Gamma(α, β) prior. After n_k observed changes over E_k item-days of exposure, the posterior predictive probability that an item confirmed a days ago still holds is

$$\hat{\rho}_k(a) = \Big(1 + \frac{a}{\beta + E_k}\Big)^{-(\alpha + n_k)}. \tag{4}$$

The class hazards are estimated from the store's own version history, so stable classes stop decaying without any tuning. (4) is a Lomax tail, a power law in a for large ages. So the same mixture argument that gives this estimator reproduces the power-law decay observed in human memory and in the environment by Anderson and Schooler [2]; see Proposition 9. When keys are missing, for example with raw conversation, a newer item with similarity above 0.8 discounts an older one (the product in Algorithm 1). A query with historical intent ("previously", "used to") disables supersession so that old versions stay reachable.

4.4 Learning

The policy keeps a ridge estimate with prior mean θ_0 fitted on development data: $\hat{\theta}_t = \theta_0 + V_t^{-1} \sum_{(x,y)} (y - x^\top \theta_0) x$ with $V_t = \lambda I + \sum x x^\top$. Feedback $y \in [0, 1]$ says whether an injected item was used. Items known to be invalid at feedback time are skipped: they say nothing about μ. The update is $O(d^2)$ by Sherman–Morrison per item and works offline. The reward signal in the plugin is execution-first: an injected item counts as used when its identifiers or paths appear in later tool calls or edits, and task success is taken from test commands, following the evidence that self-reports inflate reward

[5].

4.5 Consolidation and forgetting

Consolidation is reversible. Star clusters around a leader, with every member above a similarity threshold of 0.9 by default, are summarized extractively: density greedy on a sentence-level facility location with rate R, which yields the rate–distortion guarantee of Theorem 3. The summary is a new semantic item with parent links, the sources move to an archive tier, and EXPAND restores them on demand. This follows the evidence that repeated irreversible rewriting erodes detail [13, 70]. Forgetting moves items to the archive when they have been idle for τ days or when the active store exceeds a token capacity, evicting the lowest $\rho \cdot \text{activation} \cdot \text{usefulness}/c$ first. Both rules are opt-in, because Section 6 finds that they cost recall. Deletion happens only on an explicit user command.

4.6 Multi-resolution

An item above a token threshold can be injected as a query-biased excerpt: the sentences that cover the most query terms, up to 40 tokens, marked with the item's id. Its cost in the knapsack is the excerpt's cost, so more items fit, and the agent pulls the full text with EXPAND only when it needs it. This mode is off by default.

4.7 Complexity

With N active items, $n \le 2N_c$ candidates, f facets, embedding dimension D, and $|S| = k$ selected: a write costs $O(\log N)$ plus one embedding; candidate generation costs $O(ND)$ by brute force (or sub-linear with an ANN index) plus the FTS5 query; greedy costs $O(k\,n\,(f+n))$ per run with vectorized marginal gains, and lazy evaluation [34, 39] can reduce this further; building the similarity matrix costs $O(n^2 D)$. A bandit update costs $O(kd^2)$. Space is $O(N(D+\bar{c}))$. The memory layer itself uses no LLM tokens; the only token cost is the injected $c(S) \le B$.

5 Theory

All proofs are in Appendix A. Throughout, F is any set function with the properties of Lemma 1.

5.1 Selection

Theorem 1 (Selection guarantee)**.** *Let F be non-negative monotone submodular with $F(\emptyset) = 0$ and let $c(m) \le B$ for all candidates. The set returned by Algorithm 1 satisfies $F(S) \ge \frac{1}{2}(1 - \frac{1}{e})\,\mathrm{OPT}$.*

This is the classical cost-benefit argument [30, 34], restated with a full proof because the constant in related analyses has needed correction [58]. Better constants exist for this algorithm family, 0.405 [58], and $1 - 1/e$ with partial enumeration at higher cost [17, 56]. The worst case is pessimistic for a practical reason: at each query the following bound can be computed.

Proposition 6 (Per-query certificate)**.** *For any feasible S, with $\Delta(m \mid S) = F(S \cup \{m\}) - F(S)$,*

$$\mathrm{OPT} \le F(S) + \max_{\substack{y \in [0,1]^M \\ \sum_m y_m c(m) \le B}} \sum_{m \notin S} y_m\, \Delta(m \mid S), \quad (5)$$

and the right-hand side is computed by sorting on Δ/c in $O(n \log n)$.

5.2 Learning with expiring items

The closest results [57, 67, 69] assume that an item's expected value is a stationary linear function of its features. Memory violates this: a fact's value collapses when the fact stops holding, while its text, and so its features, stay the same. We therefore model the two factors separately.

Setting. In round $t = 1, \ldots, T$ the learner sees candidates C_t with features $\|x_{t,m}\|_2 \le 1$ (rescaled by a constant if needed), costs, and a function $F_t(S; w)$ satisfying Lemma 1 with constant L. There is an unknown $\theta^\star$ with $\|\theta^\star - \theta_0\|_2 \le \Sigma$ and $\mu_{t,m} = x_{t,m}^\top \theta^\star \in [0,1]$. Each item is valid with probability $\rho_{t,m} \in [0,1]$, which may vary arbitrarily with t, and the true weight is $w^\star_{t,m} = \rho_{t,m}\mu_{t,m}$. For each injected item the learner observes $Z_{t,m} \sim \mathrm{Bernoulli}(\rho_{t,m})$ and, if $Z_{t,m} = 1$, a reward $y_{t,m} = \mu_{t,m} + \xi_{t,m}$, where $\xi_{t,m}$ is conditionally R-sub-Gaussian. At most K items are injected per round. A validity estimate satisfies $|\hat{\rho}_{t,m} - \rho_{t,m}| \le \varepsilon_{t,m}$ for all t, m with probability at least $1 - \delta/2$.

Algorithm. Use $\bar{w}_{t,m} = \min(1, \hat{\rho}_{t,m} + \varepsilon_{t,m}) \cdot \mathrm{clip}_{[0,1]}(x_{t,m}^\top \hat{\theta}_t + \beta_t \|x_{t,m}\|_{V_t^{-1}})$, with $\hat{\theta}_t, V_t$ the prior-centred ridge estimates over the rounds where $Z = 1$, and pick S_t with an α-approximate oracle for $F_t(\cdot\,; \bar{w}_t)$ under the budget.

Theorem 2 (Validity-aware regret)**.** *Let $\lambda \ge K$ and $\beta_t = \beta_T := R\sqrt{d \log\big(4(1 + KT/\lambda)/\delta\big)} + \sqrt{\lambda}\,\Sigma$. With probability at least $1 - \delta$,*

$$\sum_{t=1}^{T} \Big[\alpha \max_{c(S) \le B} F_t(S; w_t^\star) - F_t(S_t; w_t^\star)\Big] \le L\left[2\beta_T \sqrt{2dKT \log\left(1 + \frac{KT}{d\lambda}\right)}\right.$$

The comparator is the per-round optimum under the time-varying $w_t^\star$, so the bound is dynamic; the $\rho_{t,m}$ can change arbitrarily without any variation budget, as long as they are estimated. The cost of non-stationarity is exactly the accumulated validity error. Warm starting

helps through Σ. With $\alpha = \frac{1}{2}(1 - \frac{1}{e})$ from Theorem 1 and $L = (1-\gamma)(1+\eta)$, the theorem applies to Algorithm 1.

Proposition 7 (Validity error of the hazard estimate)**.** *Suppose lifetimes of class k are i.i.d.* $\mathrm{Exp}(\lambda_k)$ *and let $\hat{\lambda}_k = n/\sum_{i \le n} L_i$ be the rate estimate from n completed lifetimes. If $n \ge 24\log(2/\delta)$ then with probability at least $1-\delta$, for all ages $a \ge 0$ simultaneously,*

$$|e^{-\hat{\lambda}_k a} - e^{-\lambda_k a}| \le \frac{3}{e}\sqrt{\frac{6\log(2/\delta)}{n}}.$$

The bound does not depend on age or on λ_k: the error is governed by the number of observed changes. So if every class that is selected keeps producing changes at rate $n_k(t) \ge \nu t$, then $\varepsilon_{t,m} = \tilde{O}(1/\sqrt{\nu t})$ and $\sum_t \sum_{m \in S_t} \varepsilon_{t,m} = \tilde{O}(K\sqrt{T/\nu})$, which gives total regret $\tilde{O}((d\sqrt{K} + K/\sqrt{\nu})\sqrt{T})$. The proposition covers the plain estimator from completed lifetimes. The implementation uses the posterior (4), which also counts censored exposure; that variant is not analyzed here.

Proposition 8 (Validity-blind learners)**.** *Let $m \ge 2$. There is an instance in which m items per round have identical features, exactly one is valid, and B fits one item. Any learner that does not observe age or version information has $\mathbb{E}[\sum_t \alpha\,\mathrm{OPT}_t - F_t(S_t)] \ge (\alpha - \frac{1}{m})T$ for every $\alpha > 1/m$. So it has linear regret, for all horizons, whereas Theorem 2 applies with $\varepsilon \equiv 0$ if the version chain is known.*

5.3 Validity and activation

Proposition 9 (Power-law survival from a hazard mixture)**.** *If hazards in a class are* $\mathrm{Gamma}(\alpha, \beta)$ *distributed, the population survival is $S(a) = (1 + a/\beta)^{-\alpha}$. As $a \to \infty$, $\log S(a) = -\alpha \log a + O(1)$, the form of ACT-R single-trace base-level activation with decay $d = \alpha$.*

5.4 Consolidation

Let X be a cluster of sentences with weights p ($\sum_x p_x = 1$), costs c, cosine similarities $s \in [0,1]$ with $s(x,x) = 1$, and let $\Phi(Z) = \sum_x p_x \max_{z \in Z} s(x,z)$, $D(Z) = 1 - \Phi(Z)$ be the distortion of a summary $Z \subseteq X$.

Theorem 3 (Rate–distortion for extractive consolidation)**.** *Let Z_ℓ be the first ℓ picks of density greedy on Φ, with rate $R_\ell = c(Z_\ell)$. For every summary $Z^\star$ of rate $R^\star$,*

$$D(Z_\ell) \le e^{-R_\ell/R^\star} + \big(1 - e^{-R_\ell/R^\star}\big) D(Z^\star).$$

Moreover, for every query vector e_q and every ℓ_u-Lipschitz item score $u_q(x) = \phi(\langle e_q, e_x \rangle)$,

$$\mathbb{E}_{x \sim p}\big[u_q(x) - u_q(z(x))\big] \le \ell_u \sqrt{2\,D(Z_\ell)}, \qquad z(x) = \arg\max_{z \in Z_\ell} s(x,z).$$

The token saving is $c(X) - R_\ell$, so distortion decays exponentially in rate against any reference summary. The bound averages over p and says nothing about a rare needle in the cluster. That is why consolidation here keeps the sources: if the summary is retrieved, Expand recovers any source at the cost of its tokens.

5.5 Store size

Theorem 4 (Store stability)**.** *Let items arrive as a Poisson process of rate a, and let each be accessed by an independent Poisson process of rate r, drawn i.i.d. from a distribution with $\mathbb{E}[(e^{r\tau} - 1)/r] < \infty$. An item is archived after an idle gap of τ. Then an item's active lifetime has mean $(e^{r\tau} - 1)/r$ given r, and the number of active items converges in distribution to* $\mathrm{Poisson}\big(a\,\mathbb{E}[(e^{r\tau} - 1)/r]\big)$.

Without forgetting, the store grows linearly in t. With an idle rule, the active store is bounded in distribution, and the retained set is biased exponentially toward frequently used items. The result assumes independent access processes; real access patterns are bursty, so Section 6 measures store size directly.

6 Experiments

6.1 Setup

Mode. No LLM was available, so all metrics are retrieval-level. Hypotheses about end-to-end effects are out of scope.

Benchmarks. *LongMemEval-S* [64]: 500 questions, each with its own store of about 490 turns; 370 test questions remain after dropping those without flagged evidence and the dev split. *LoCoMo* [37]: 10 long conversations; conversations 0–2 are dev, leaving 1483 test questions. *RepoMem* (new): eight Python repositories at 14 snapshots over six years. Each interval is a session in which the simulated agent records commit notes and facts observed in changed files. Facts (constants, signatures, symbol locations, dependency constraints) come from `ast`, so a fact is stale exactly when the same key held another value at an earlier snapshot. Fact queries ask for the current value; task queries use a real later commit subject, with earlier commits on the same files as gold. Two repositories are dev, six test (4322 queries over three seeded streams each). We run RepoMem with slot keys, as a host agent would supply, and without. Four more repositories are a holdout (Section 6.5).

Baselines. No memory (recall 0), full history (recall 1 at the cost of the whole store, a mean of 107,994 tokens per question on LongMemEval-S and 26,549 on LoCoMo), recency, BM25 (FTS5), dense (all-MiniLM-L6-v2 [61]), hybrid RRF [14], hybrid followed by a

cross-encoder reranker, MMR [8], Generative Agents scoring [45] with constant importance, facility location under a knapsack as in PACMS [19], hybrid top-k for $k \in \{3, 5, 10, 20\}$, and hybrid with superseded versions removed (temporal invalidation alone, in the spirit of [49]). All methods share candidate generators, token counts (`cl100k_base`) and an 8-token per-item overhead, and every ranking is filled greedily to the budget, which already makes the baselines budget-aware. Mem0 2.2.1 with `infer=False` and local Qdrant ran on a random 40-question sample. Systems that need an LLM on write were not run.

Protocol. The prior θ_0 and hyperparameters were set on dev splits only. Metrics: evidence recall, tokens injected, staleness error (an outdated version is injected and either the current one is absent or ranks lower), latency and store size. Retrieval is deterministic; seeds govern RepoMem streams and online runs (five seeds). CIs are paired bootstrap over questions; for clustered data we also give a cluster bootstrap over streams, which is the more honest interval. p-values are Wilcoxon signed-rank with Holm correction per dataset.

6.2 Recall against tokens

Figure 1 shows the fronts and Table 2 the values. On LongMemEval-S, where turn lengths vary widely, KANERVA dominates every baseline at every budget from 256 up: at 1024 tokens recall is 0.860 against 0.769 for BM25, 0.761 for hybrid and 0.870 for the reranker. The ablation shows where this comes from: without cost density it falls to 0.784. On LoCoMo, where turns are short and similar in length, KANERVA leads up to 1024 tokens (0.722 against 0.682) and then loses to hybrid: 0.842 against 0.873 at 4096. On RepoMem without keys it trails hybrid in recall by up to about two points (0.890 against 0.908). Dense retrieval is better than KANERVA at the smallest budget there. Lexical-only KANERVA, the zero-dependency default of the plugin, stays close on LongMemEval-S (0.820) and below full KANERVA elsewhere.

On the Mem0 sample (40 questions, mixed categories), Mem0's verbatim mode reached 0.925 at 1024 tokens against 0.975 for KANERVA and 0.875 for hybrid on the same questions. Its write took 18 ms per turn.

Table **??** gives the paired differences against the strongest rival at three budgets. The cluster intervals on LoCoMo and RepoMem are wide because they rest on few conversations or repositories, so differences of a point or two there should be read as ties.

6.3 Staleness

Table **??** is the main result on RepoMem. Over all queries, stale error is about a third for similarity-based methods at any budget, and restricted to facts that changed it is 0.700 for hybrid at 1024 tokens: more often than not the agent would read an outdated value ahead of the current one, or instead of it. A larger budget does not help. With keys, KANERVA reaches 0.003. Without keys, where it has only similarity and timestamps, it reaches 0.016 overall, or 0.030 on changed facts. Hybrid with invalidation also reaches zero with keys, and that method has no answer without them, since its filter needs them. Removing validity from KANERVA brings the error back in full, so the component is responsible. The price is the recall gap described above, because older versions are pushed out even for task queries that would have benefited from them.

6.4 Ablations

Table **??** removes one component at a time. Cost density is decisive on LongMemEval-S and slightly harmful on LoCoMo at small budgets, where *removing* it adds up to 0.023 recall, because uniform turn lengths give density nothing to exploit and short turns get an edge. Validity costs a little recall on chat benchmarks and pays for itself on RepoMem through staleness, which recall does not show. Facets help at small budgets. The facility-location term and activation have effects indistinguishable from zero on recall; we keep them because they are cheap, and because activation becomes informative only with real access histories, which these benchmarks lack. An honest reading is that this benchmark set does not establish their value.

6.5 A disclosed post-hoc change

The first test run showed KANERVA below hybrid on keyed RepoMem, and the ablation pointed at soft supersession: signatures of different functions are near-duplicates, so they discounted each other. Since keyed items have exact version chains, we restricted soft supersession to items without a key. This fix was found on test data, so numbers on those six repositories are no longer clean. Table 2 reports both the first-run rule and the fix. To check it fairly we added four repositories after that run (starlette, typer, black, aiohttp), never used for tuning. There, at 1024 tokens, the fixed method has recall 0.834 against 0.818 for the first-run rule, 0.828 for hybrid and 0.822 for dense, with stale error 0.001 against 0.341 for hybrid. LongMemEval and LoCoMo have no keys, so the change does not touch them.

6.6 Online learning

Feedback here is simulated from evidence labels, as described in Section 7. Each seed shuffles the order of work and shares a single policy; Table 6 reports recall

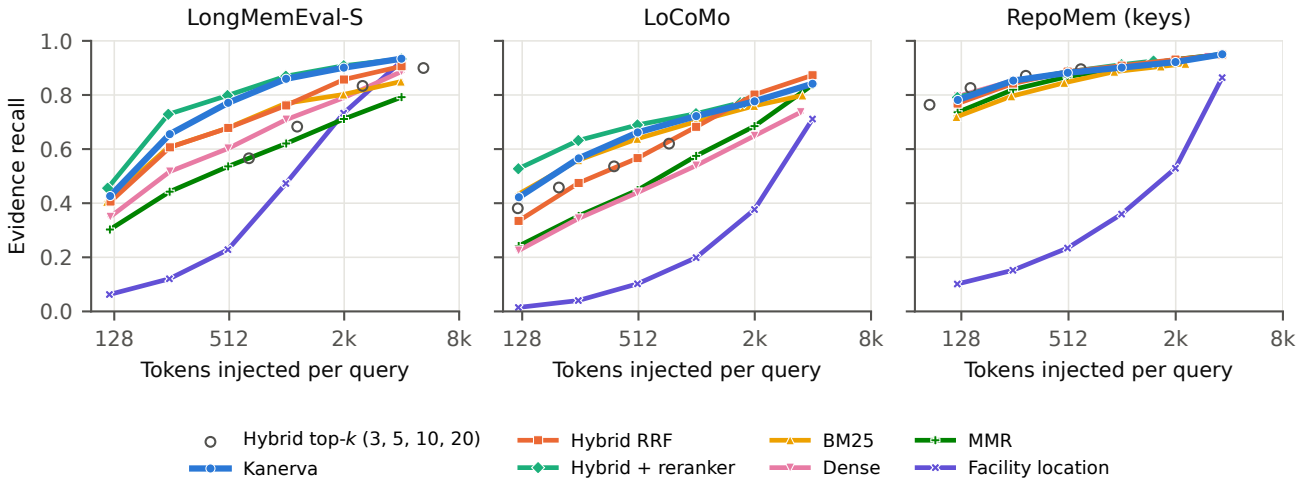


Figure 1: Evidence recall against mean tokens injected. Each curve visits budgets 128 to 4096; open circles are fixed top-k. Up and to the left is better.

at $B = 512$ over the second half of each run, and Figure 2 the learning curves. TBD

6.7 Checks of the theory on synthetic instances

With known ground truth (eight seeds, horizon 4000, 40 items per round, classes with different hazards), cumulative regret against the per-round greedy optimum is 23 for a learner that knows validity, 174 when it estimates hazards and 1997 when it is blind. The log–log slopes of regret in T are 0.28, 0.74 and 0.97. Theorem 2 predicts a slope near $\frac{1}{2}$ for the first two up to log factors, and Proposition 8 predicts a slope of 1 in the adversarial case. Over 1600 random selection problems, the mean of $F(S)$ over the certificate upper bound was 0.707, and the minimum 0.600, far above the worst-case constant of 0.316.

6.8 Consolidation, forgetting and multi-resolution

Consolidation. Table **??** applies consolidation at several thresholds. A gold item counts as found if its summary is selected, which measures discoverability, since the sources stay expandable. TBD The rate–distortion bound of Theorem 3 held in all 2100 cases with $Z^\star = X$ (0 violations). Mean distortion falls from 0.756 at rate 0.1 to 0.369 at 0.5.

Forgetting. On RepoMem streams, Table **??** compares keeping everything with idle-time rules and token capacities. TBD

Multi-resolution. Injecting query-biased excerpts of long items, with a pointer to the full text, raises the number of items that fit. On LongMemEval-S at 512 tokens, turn reachability moves from 0.771 with full items to 0.849 with excerpts above 48 tokens, and at 1024 tokens from 0.860 to 0.909. This counts an excerpt as reaching the turn. Whether the excerpt contains the answer, or the agent would expand it, was not measured without an LLM.

6.9 Latency and scale

On an M1 Pro, at 10^4 items, hybrid retrieval takes 53.5 ms at the median and 65.3 ms at the 95th percentile, and lexical retrieval 27.7 ms and 36.6 ms. At 10^5 items these are 85.1 and 52.8 ms. A write costs 0.34 ms, excluding the embedding. The memory layer spends no LLM tokens, so its token cost is the injected context alone.

7 Limitations

No LLM in the evaluation. Every metric is retrieval-level. We do not know how much recall per token translates into correct answers or finished tasks. Evidence recall treats a turn as all-or-nothing, and a selected turn may still be hard for a model to use. Answer accuracy with an LLM is the obvious next measurement.

Baselines that need an LLM were not run. Mem0 with inference, Graphiti, A-MEM, HippoRAG, LangMem and Cognee all call an LLM on write. They could not run as designed, so the comparison does not cover them. Mem0 ran only in its verbatim mode

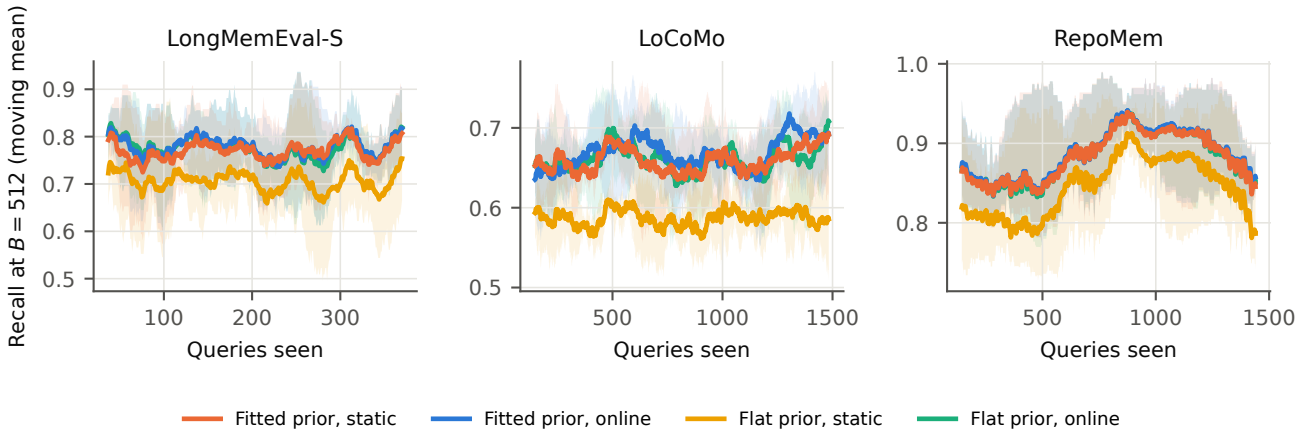


Figure 2: Online learning curves: moving-mean recall at $B = 512$. Bands span the seeds.

(`infer=False`), on a subset. Our comparison says nothing about what LLM extraction would add to those systems, and published evidence suggests a small effect on the write side [68].

Feedback was simulated. The online experiments derive rewards from evidence labels. Real implicit signals, such as an item's identifiers reappearing in tool calls, are noisier and biased in ways that label flipping does not capture. The plugin's signal has not been validated against real sessions.

RepoMem is a proxy. Queries come from templates over extracted facts, so they contain identifiers and favor lexical search; real prompts are vaguer. The agent's notes are simulated from code and commit history. Every changed fact is assumed to be observed, so a store never misses a change. The task track uses file overlap as relevance, which is a weak label. Keyed mode gets perfect slot keys from the static analyzer, so it is an upper bound on what a real write path provides; plain mode is the conservative end. All eight repositories are Python libraries.

The prior was fitted on the same benchmark families. θ_0 and the selection parameters come from dev splits of the three benchmarks, so test performance on a new domain may be lower. Queries in a LongMemEval split share a distractor pool across questions.

Theory versus code. Theorem 2 assumes a linear μ, conditionally independent rewards and validity revealed with feedback. In practice validity is seen only when a successor appears. The theorem needs the UCB bonus and the validity margin ε; the shipped defaults use $\beta = 0$ and $\varepsilon = 0$, and a small weight floor, so the guarantee covers the analyzed variant, not the default. Proposition 7 covers the estimator from completed lifetimes, not the censored posterior in the code. The worst-case constant is far from what the certificate shows in practice, and the certificate bounds the surrogate F, not recall. Facets are bag-of-terms, so paraphrases only reach them through the query facet.

Token counts. We count with `cl100k_base` as a proxy for Claude's tokenizer, and the 8-token item overhead is a fixed assumption.

Plugins. The plugins were tested with simulated hook payloads, the real entry script, and the plugin validator, but not in live Claude Code or OpenCode sessions.

8 Conclusion

Treating agent memory as budgeted selection over items with explicit validity gives a read path that needs no LLM, states its own optimality gap per query, and carries a regret guarantee that survives expiring items. TBD

The next step is an end-to-end study: whether higher recall per token, and fewer stale injections, change what a coding agent does. The retrieval results here are necessary for that outcome and do not establish it.

Reproducibility. `make data && make reproduce` rebuilds every result in `results/*.json` with commit hashes and seeds, and `make check` verifies each number in this paper against those files. The RepoMem manifest pins every repository snapshot.

References

[1] Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved algorithms for linear stochastic bandits. In *Advances in Neural Information Processing Systems 24*, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/e1d5be1c7f2f456670de3d53c7b54f4a-Abstract.html. Accessed 2026-10-30.

[2] John R. Anderson and Lael J. Schooler. Reflections of the environment in memory. *Psychological Science*, 2:396–408, 1991. doi: 10.1111/j.1467-9280.1991.tb00174.x. URL https://doi.org/10.1111/j.1467-9280.1991.tb00174.x. Accessed 2026-10-30.

[3] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111:1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036. URL https://doi.org/10.1037/0033-295X.111.4.1036. Accessed 2026-10-30.

[4] Anthropic. How claude remembers your project, 2026. URL https://code.claude.com/docs/en/memory. Official docs. Accessed 2026-10-30.

[5] Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory reward inflation in self-improving llm agents, 2026. URL https://arxiv.org/abs/2608.00017. Accessed 2026-10-30.

[6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2024. URL https://arxiv.org/abs/2501.00663. Accessed 2026-10-30.

[7] Vincent-Pierre Berges, Barlas Ouz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale, 2024. URL https://arxiv.org/abs/2412.09764. Accessed 2026-10-30.

[8] Jaime Carbonell and Jade Goldstein. The use of mmr, diversity-based reranking for reordering documents and producing summaries. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025. Accessed 2026-10-30.

[9] Wei Chen, Yajun Wang, and Yang Yuan. Combinatorial multi-armed bandit: General framework and applications. In *International Conference on Machine Learning*, volume 28 of *PMLR*, pages 151–159, 2013. URL https://proceedings.mlr.press/v28/chen13a.html. Accessed 2026-10-30.

[10] Wei Chen, Yajun Wang, Yang Yuan, and Qinshi Wang. Combinatorial multi-armed bandit and its extension to probabilistically triggered arms, 2014. URL https://arxiv.org/abs/1407.8339. Accessed 2026-10-30.

[11] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts, 2023. URL https://arxiv.org/abs/2305.14788. Accessed 2026-10-30.

[12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL https://arxiv.org/abs/2504.19413. Accessed 2026-10-30.

[13] Ashwin Gerard Colaco and Nada Lahjouji. What to keep, what to forget: A rate-distortion view of memory compaction in llms and agents, 2026. URL https://arxiv.org/abs/2607.08032. Accessed 2026-10-30.

[14] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114. Accessed 2026-10-30.

[15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2024. URL https://arxiv.org/abs/2404.16130. Accessed 2026-10-30.

[16] Liyang Fan, Yingcheng Shi, Yongbin Li, Chenghao Sun, Xin Chen, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. Vibemembench: Evaluating memory systems for coding agents on real repository coding tasks, 2026. URL https://arxiv.org/abs/2609.23570. Accessed 2026-10-30.

[17] Moran Feldman, Zeev Nutov, and Elad Shoham. Practical budgeted submodular maximization, 2020. URL https://arxiv.org/abs/2007.04937. Accessed 2026-10-30.

[18] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model, 2023. URL https://arxiv.org/abs/2307.06945. Accessed 2026-10-30.

[19] Manu Ghulyani, Arunabh Singh, Karan Bharadwaj, Ankit Nath, and Suranjan Goswami. Pacms: Submodular context selection as a pluggable engine for llm agents, 2026. URL https://arxiv.org/abs/2606.20047. Accessed 2026-10-30.

[20] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation, 2024. URL https://arxiv.org/abs/2410.05779. Accessed 2026-10-30.

[21] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From rag to memory: Non-parametric continual learning for large language models. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.14802. Accessed 2026-10-30.

[22] Yudai Honda, Keita Fujita, Keiichi Zempo, and Shogo Fukushima. Human-like remembering and forgetting in llm agents: An act-r-inspired memory architecture. In *International Conference on Human-Agent Interaction*, pages 229–237, 2025. doi: 10.1145/3765766.3765803. URL https://doi.org/10.1145/3765766.3765803. Accessed 2026-10-30.

[23] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2507.05257. Year refers to the ICLR 2026 publication. Accessed 2026-10-30.

[24] Mehmet Iscan. Pythalab-mera: Validation-grounded memory, retrieval, and acceptance control for frozen-llm coding agents, 2026. URL https://arxiv.org/abs/2605.08468. Accessed 2026-10-30.

[25] Mehmet Iscan. Learning when to remember: Risk-sensitive contextual bandits for abstention-aware memory retrieval in llm-based coding agents, 2026. URL https://arxiv.org/abs/2604.27283. Accessed 2026-10-30.

[26] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models, 2023. URL https://arxiv.org/abs/2310.05736. Accessed 2026-10-30.

[27] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression, 2023. URL https://arxiv.org/abs/2310.06839. Accessed 2026-10-30.

[28] Prakhar Khatri. Do context files help coding agents? a two-agent ablation study on real repositories, 2026. URL https://arxiv.org/abs/2607.27250. Accessed 2026-10-30.

[29] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70: 39–45, 1999. doi: 10.1016/S0020-0190(99)00031-9. URL https://doi.org/10.1016/S0020-0190(99)00031-9. Accessed 2026-10-30.

[30] Andreas Krause and Carlos Guestrin. A note on the budgeted maximization of submodular functions. Technical Report CMU-CALD-05-103, Carnegie Mellon University, 2005. URL http://reports-archive.adm.cs.cmu.edu/anon/cald/CMU-CALD-05-103.pdf. Accessed 2026-10-30.

[31] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning, 2012. URL https://arxiv.org/abs/1207.6083. Accessed 2026-10-30.

[32] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20:512–534, 2016. doi: 10.1016/j.tics.2016.05.004. URL https://doi.org/10.1016/j.tics.2016.05.004. Accessed 2026-10-30.

[33] Branislav Kveton, Zheng Wen, Azin Ashkan, and Csaba Szepesvari. Tight regret bounds for stochastic combinatorial semi-bandits, 2014. URL https://arxiv.org/abs/1410.0949. Accessed 2026-10-30.

[34] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 420–429, 2007. doi: 10.1145/1281192.1281239. URL https://doi.org/10.1145/1281192.1281239. Accessed 2026-10-30.

[35] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents, 2026. URL https://arxiv.org/abs/2602.05892. Accessed 2026-10-30.

[36] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation, 2024. URL https://arxiv.org/abs/2404.14469. Accessed 2026-10-30.

[37] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.17753. Accessed 2026-10-30.

[38] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102:419–457, 1995. doi: 10.1037/0033-295X.102.3.419. URL https://doi.org/10.1037/0033-295X.102.3.419. Accessed 2026-10-30.

[39] Michel Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, Lecture Notes in Control and Information Sciences, pages 234–243. 1978. doi: 10.1007/BFb0006528. URL https://doi.org/10.1007/BFb0006528. Accessed 2026-10-30.

[40] Kazuki Nakayashiki. When stale constraints go unchecked: Budgeted verification failures in inherited agent memory, 2026. URL https://arxiv.org/abs/2608.25553. Accessed 2026-10-30.

[41] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971. URL https://doi.org/10.1007/BF01588971. Accessed 2026-10-30.

[42] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL https://arxiv.org/abs/2509.25140. Accessed 2026-10-30.

[43] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. URL https://arxiv.org/abs/2310.08560. Accessed 2026-10-30.

[44] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression, 2024. URL https://arxiv.org/abs/2403.12968. Accessed 2026-10-30.

[45] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL https://arxiv.org/abs/2304.03442. UIST 2023; DOI 10.1145/3586183.3606763. Accessed 2026-10-30.

[46] Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. Adagres:adaptive greedy context selection via redundancy-aware scoring for token-budgeted rag, 2025. URL https://arxiv.org/abs/2512.25052. Accessed 2026-10-30.

[47] Aleksandar Petrov, Mark Sandler, Andrey Zhmoginov, Nolan Miller, and Max Vladymyrov. Long context in-context compression by getting to the gist of gisting, 2025. URL https://arxiv.org/abs/2504.08934. Accessed 2026-10-30.

[48] Nathanaël Carraz Rakotonirina, Mohammed Hamdy, Jon Ander Campos, Lucas Weber, Alberto Testoni, Marzieh Fadaee, Sandro Pezzelle, and Marco Del Tredici. From tools to teammates: Evaluating llms in multi-session coding interactions, 2025. URL https://arxiv.org/abs/2502.13791. Accessed 2026-10-30.

[49] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL https://arxiv.org/abs/2501.13956. Accessed 2026-10-30.

[50] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. Okapi at trec-3. In *Text REtrieval Conference (TREC-3)*, pages 109–126, 1994. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/okapi_trec3.pdf. Accessed 2026-10-30.

[51] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval, 2024. URL https://arxiv.org/abs/2401.18059. Accessed 2026-10-30.

[52] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366. Accessed 2026-10-30.

[53] Baixi Sun, Le Chen, Anjir Ahmed Chowdhury, Xiaolong Ma, Chih-Hsuan Yang, Mingze Xia, Syed Zawad, Sheng Di, Rajkumar Kettimuthu, Huihuo Zheng, Rajeev Thakur, Venkatram Vishwanath, and Feng Yan. Lam: Efficient lossy agent memory framework with a retrieval-score error bound, 2026. URL https://arxiv.org/abs/2609.32256. Accessed 2026-10-30.

[54] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts, 2019. URL https://arxiv.org/abs/1909.13231. Accessed 2026-10-30.

[55] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states, 2024. URL https://arxiv.org/abs/2407.04620. Accessed 2026-10-30.

[56] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32:41–43, 2004. doi: 10.1016/S0167-6377(03)00062-2. URL https://doi.org/10.1016/S0167-6377(03)00062-2. Accessed 2026-10-30.

[57] Sho Takemori, Masahiro Sato, Takashi Sonoda, Janmajay Singh, and Tomoko Ohkuma. Submodular bandit problem under multiple constraints, 2020. URL https://arxiv.org/abs/2006.00661. PMLR 124 (UAI 2020). Accessed 2026-10-30.

[58] Jing Tang, Xueyan Tang, Andrew Lim, Kai Han, Chongshou Li, and Junsong Yuan. Revisiting modified greedy algorithm for monotone submodular maximization with a knapsack constraint, 2020. URL https://arxiv.org/abs/2008.05391. Accessed 2026-10-30.

[59] Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. Memconflict: Evaluating long-term memory systems under memory conflicts, 2026. URL https://arxiv.org/abs/2605.20926. Accessed 2026-10-30.

[60] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in llms. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.27246. Accessed 2026-10-30.

[61] Sentence Transformers. sentence-transformers/all-minilm-l6-v2 model card, 2026. URL https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Model card. Accessed 2026-10-30.

[62] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291. Accessed 2026-10-30.

[63] Zheng Wen, Branislav Kveton, and Azin Ashkan. Efficient learning in large-scale combinatorial semi-bandits, 2014. URL https://arxiv.org/abs/1406.7443. Accessed 2026-10-30.

[64] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10813. Accessed 2026-10-30.

[65] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. URL https://arxiv.org/abs/2309.17453. Accessed 2026-10-30.

[66] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents, 2025. URL https://arxiv.org/abs/2502.12110. Accessed 2026-10-30.

[67] Baosheng Yu, Meng Fang, and Dacheng Tao. Linear submodular bandits with a knapsack constraint. In *AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi: 10.1609/aaai.v30i1.10154. URL https://doi.org/10.1609/aaai.v30i1.10154. Accessed 2026-10-30.

[68] Boqin Yuan, Yue Su, and Kun Yao. Diagnosing retrieval vs. utilization bottlenecks in llm agent memory, 2026. URL https://arxiv.org/abs/2603.02473. Accessed 2026-10-30.

[69] Yisong Yue and Carlos Guestrin. Linear submodular bandits and their application to diversified retrieval. In *Advances in Neural Information Processing Systems 24*, pages 2483–2491, 2011. URL https:

//proceedings.neurips.cc/paper/2011/hash/33ebd5b07dc7e407752fe773eed20635-Abstract.html. Accessed 2026-10-30.

[70] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2025. URL https://arxiv.org/abs/2510.04618. Accessed 2026-10-30.

[71] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, Bo Tang, and Muning Wen. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory, 2026. URL https://arxiv.org/abs/2601.03192. Accessed 2026-10-30.

[72] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. URL https://arxiv.org/abs/2306.14048. Accessed 2026-10-30.

[73] Ziyi Zhang, Shuang Cui, Haotian Zhang, and Xiaoyu Wang. Pc-submax: Efficient prompt compression via regularized submodular maximization, 2026. URL https://arxiv.org/abs/2609.32474. Accessed 2026-10-30.

[74] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL https://arxiv.org/abs/2602.08316. Accessed 2026-10-30.

[75] Mingxi Zou, Zhihan Guo, Langzhang Liang, Zhuo Wang, Qifan Wang, Qingsong Wen, Irwin King, Lizhen Qu, and Zenglin Xu. Remember the decision, not the description: A rate-distortion framework for agent memory, 2026. URL https://arxiv.org/abs/2605.10870. Accessed 2026-10-30.

Table 2: Evidence recall at each token budget on test splits. Bold marks the best method per column within a dataset.

Method	128	256	512	1024	2048	4096
LongMemEval-S						
KANERVA	0.426	0.655	0.771	0.860	0.901	**0.934**
KANERVA, first run	0.426	0.655	0.771	0.860	0.901	**0.934**
KANERVA, lexical	0.406	0.625	0.746	0.820	0.885	0.914
Hybrid + reranker	**0.456**	**0.729**	**0.798**	**0.870**	**0.908**	0.933
Hybrid RRF	0.406	0.607	0.678	0.761	0.857	0.906
BM25	0.405	0.604	0.677	0.769	0.802	0.850
Dense	0.350	0.517	0.602	0.709	0.790	0.887
MMR	0.303	0.443	0.536	0.621	0.711	0.792
Facility location	0.062	0.121	0.228	0.473	0.733	0.923
Generative Agents	0.223	0.295	0.345	0.406	0.454	0.521
Recency	0.000	0.003	0.005	0.018	0.029	0.055
LoCoMo						
KANERVA	0.422	0.566	0.662	0.722	0.777	0.842
KANERVA, first run	0.422	0.566	0.662	0.722	0.777	0.842
KANERVA, lexical	0.429	0.549	0.635	0.697	0.764	0.798
Hybrid + reranker	**0.527**	**0.632**	**0.689**	**0.731**	0.771	0.773
Hybrid RRF	0.335	0.475	0.567	0.682	**0.802**	**0.873**
BM25	0.433	0.559	0.638	0.702	0.760	0.798
Dense	0.225	0.344	0.439	0.539	0.649	0.736
MMR	0.242	0.353	0.448	0.575	0.685	0.837
Facility location	0.015	0.040	0.102	0.199	0.377	0.711
Generative Agents	0.024	0.043	0.061	0.131	0.302	0.601
Recency	0.002	0.003	0.014	0.035	0.082	0.148
RepoMem, keys						
KANERVA	0.782	**0.853**	0.882	0.901	0.922	**0.950**
KANERVA, first run	0.757	0.816	0.852	0.879	0.913	**0.951**
KANERVA, lexical	0.735	0.809	0.861	0.887	0.908	0.915
Hybrid + reranker	0.791	0.850	0.886	**0.912**	0.925	0.925
Hybrid RRF	0.768	0.842	**0.886**	0.908	**0.931**	0.949
Hybrid, live only	0.781	0.823	0.846	0.869	0.892	0.908
BM25	0.719	0.795	0.845	0.886	0.905	0.915
Dense	**0.795**	**0.853**	0.881	0.904	0.926	0.941
MMR	0.736	0.820	0.868	0.899	**0.931**	**0.950**
Facility location	0.101	0.152	0.234	0.360	0.529	0.864
Generative Agents	0.194	0.290	0.496	0.666	0.868	0.930
Recency	0.008	0.018	0.038	0.088	0.188	0.325
RepoMem, no keys						
KANERVA	0.771	0.835	0.866	0.890	0.913	**0.950**
KANERVA, first run	0.771	0.835	0.866	0.890	0.913	**0.950**
KANERVA, lexical	0.724	0.795	0.841	0.876	0.904	0.915
Hybrid RRF	0.769	0.842	**0.886**	**0.908**	**0.931**	0.949
Hybrid, live only	0.769	0.842	**0.886**	**0.908**	**0.931**	0.949
BM25	0.720	0.795	0.845	0.886	0.905	0.915
Dense	**0.795**	**0.853**	0.881	0.904	0.926	0.941
MMR	0.738	0.820	0.868	0.899	**0.931**	**0.950**
Facility location	0.103	0.156	0.239	0.368	0.540	0.873
Generative Agents	0.192	0.289	0.496	0.665	0.868	0.931
Recency	0.008	0.017	0.038	0.087	0.186	0.321
RepoMem holdout, keys						
KANERVA	**0.662**	**0.740**	**0.800**	**0.834**	0.863	0.905
KANERVA, first run	0.638	0.712	0.777	0.818	0.855	0.906
KANERVA, lexical	0.623	0.699	0.772	0.824	0.861	0.887
Hybrid RRF	0.654	0.730	0.788	0.828	**0.869**	**0.908**
Hybrid, live only	0.660	0.729	0.778	0.810	0.851	0.886
BM25	0.604	0.685	0.745	0.794	0.840	0.884
Dense	0.651	0.736	0.783	0.822	0.866	0.903
MMR	0.614	0.702	0.752	0.800	0.851	0.904
Facility location	0.121	0.164	0.249	0.365	0.499	0.745
Generative Agents	0.097	0.130	0.253	0.570	0.797	0.905
Recency	0.003	0.008	0.031	0.060	0.096	0.206

Table 6: Prequential recall at $B = 512$ over the second half of each run, mean ± s.d. over five seeds. "Noisy" flips 20% of rewards. "Flat" starts from a uniform prior.

Dataset	Static	Online	UCB	Noisy	Debiased	Flat static	Flat online
LongMemEval-S	0.772 ± 0.024	0.775 ± 0.025	0.772 ± 0.020	0.756 ± 0.014	—	0.706 ± 0.023	0.769 ± 0.020
LoCoMo	0.664 ± 0.018	0.672 ± 0.016	0.675 ± 0.013	0.608 ± 0.013	—	0.588 ± 0.008	0.665 ± 0.015
RepoMem	0.897 ± 0.022	0.902 ± 0.021	0.902 ± 0.021	0.856 ± 0.021	—	0.860 ± 0.026	0.899 ± 0.020

A Proofs

Proof of Lemma 1. For a facet f, $g_f(S) = 1 - \prod_{m \in S}(1 - u_m)$ with $u_m = w(m)a_{mf} \in [0, 1]$. Adding m to S gains $u_m \prod_{n \in S}(1-u_n)$, which is non-negative and non-increasing in S. So g_f is monotone submodular. The additive term is modular with non-negative coefficients. Each term $S \mapsto \max_{m \in S} s(c, m)$ (with $\max \emptyset = 0$) is the facility-location function, which is monotone submodular. Non-negative combinations preserve both properties, and $F(\emptyset) = 0$.

Coordinate monotonicity follows because $\prod(1 - w(m)a_{mf})$ decreases in each $w(m)$ and the third term does not depend on w. For the Lipschitz bound, $|\prod_i(1 - u_i) - \prod_i(1 - v_i)| \le \sum_i |u_i - v_i|$ for $u, v \in [0, 1]$ by telescoping. So each g_f moves by at most $\sum_{m \in S} |w(m) - w'(m)|$ as $a_{mf} \le 1$, and $\sum_f \omega_f = 1$. The additive term moves by at most η times the same sum, and the factor $(1 - \gamma)$ gives L. □

Proof of Proposition 1. Take additive U. One item a has score $1 + \epsilon$ and cost B. There are $\lfloor B/c_{\min} \rfloor$ items of score 1 and cost $c_{\min}$. The rule fills with a first and then has no budget, so it earns $1 + \epsilon$ against $\mathrm{OPT} \ge \lfloor B/c_{\min} \rfloor$. Let $\epsilon \to 0$. □

Proof of Proposition 2. Let there be k facets of equal weight. Facet 1 has k duplicates, each with modular score $1 + \epsilon$. Each other facet has one item with score 1. All costs equal 1, and $B = k$. A modular rule takes the k duplicates and covers one facet; the optimum takes one item per facet and covers all k. □

Proof of Proposition 3. Recency selects only from the $\lfloor B/c_{\min} \rfloor$ newest items, which exclude the item in question. A relevance rule does not depend on the items' order in time. □

Proof of Proposition 4. Take two versions with equal text similarity to the query, and let the assignment of the two timestamps be uniform. An invariant score cannot depend on the assignment, so the ranking between the two is independent of which one is current. The outdated version ranks first with probability 1/2, or at least 1/2 under ties broken independently of time. □

Proof of Proposition 5. Let each round have two items with features e_1, e_2, costs 1, $B = 1$, $\rho \equiv 1$ and additive F. For a fixed θ, assume without loss of generality that $\theta_1 \ge \theta_2$; the fixed policy injects e_1 on every round. Take $\theta^\star = (0, 1)$, which is valid since $\mu \in [0, 1]$. The policy earns 0 per round against 1, so its α-regret is αT for each $\alpha > 0$. Theorem 2 with $\varepsilon \equiv 0$ gives $\tilde{O}(\sqrt{T})$ on this instance. □

Proof of Theorem 1. Let O be an optimal set and consider the density run ($\varrho = 1$) as a sequence of considered items. Let $g_1, g_2, \dots$ be the items accepted, in order, and write $G_i = \{g_1, \dots, g_i\}$. If no element of O is rejected, then every element of O is either accepted or left with zero marginal gain at termination, so $F(G) \ge F(G \cup O) \ge F(O)$ and we are done. Otherwise let $e^\star$ be the first element of O rejected, and let ℓ be the number of accepted items before it, so $c(G_\ell) + c(e^\star) > B$.

For $i \le \ell$, when g_i was chosen, every element of $O \setminus G_{i-1}$ was still available, since none had been rejected. By monotonicity and submodularity,

$$F(O) \le F(O \cup G_{i-1}) \le F(G_{i-1}) + \sum_{o \in O \setminus G_{i-1}} \Delta(o \mid G_{i-1}) \le F(G_{i-1}) + B\,\frac{\Delta(g_i \mid G_{i-1})}{c(g_i)}.$$

The same inequality holds with $e^\star$ in the role of $g_{\ell+1}$ and $G_{\ell+1} := G_\ell \cup \{e^\star\}$, $c(g_{\ell+1}) := c(e^\star)$, since $e^\star$ had maximum density when chosen. Set $\delta_i = F(O) - F(G_i)$. The inequality reads $\delta_i \le (1 - c(g_i)/B)\,\delta_{i-1}$. If some $\delta_i \le 0$ with $i \le \ell$ then $F(G) \ge F(O)$ and we are done. Otherwise, using $0 \le 1 - x \le e^{-x}$ on $x = c(g_i)/B \in (0, 1]$,

$$\delta_{\ell+1} \le F(O) \prod_{i=1}^{\ell+1} \Big(1 - \frac{c(g_i)}{B}\Big) \le F(O) \exp\Big(-\frac{\sum_{i \le \ell+1} c(g_i)}{B}\Big) \le \frac{F(O)}{e},$$

because $\sum_{i \le \ell+1} c(g_i) = c(G_\ell) + c(e^\star) > B$. So $F(G_\ell \cup \{e^\star\}) \ge (1 - 1/e)F(O)$. By subadditivity $F(G_\ell) + F(\{e^\star\}) \ge (1 - 1/e)F(O)$, so one of the two terms is at least $\frac{1}{2}(1 - 1/e)F(O)$. Now $F(G) \ge F(G_\ell)$ by monotonicity, and $F(\{e^\star\})$ is at most the value of the best singleton, which fits since $c(e^\star) \le B$. The algorithm returns the maximum over a set of runs that includes both. □

Proof of Proposition 6. For any feasible O, monotonicity and submodularity give $F(O) \le F(S) + \sum_{m \in O \setminus S} \Delta(m \mid S)$. The indicator of $O \setminus S$ is feasible for the linear program, whose optimum is reached by the fractional-knapsack rule. □

Lemma 2 (Batched elliptical potential)**.** *Let $\lambda \geq K$, $V_1 = \lambda I$, $V_{t+1} = V_t + \sum_{i \in A_t} x_i x_i^\top$ with $|A_t| \leq K$ and $\|x_i\| \leq 1$. Then $\sum_{t \leq T} \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \leq 2d \log(1 + KT/(d\lambda))$.*

Proof. Let $B_t = V_t^{-1/2}(\sum_{i \in A_t} x_i x_i^\top)V_t^{-1/2}$, positive semidefinite with $\operatorname{tr} B_t = \sum_{i \in A_t} \|x_i\|^2_{V_t^{-1}} \leq K/\lambda \leq 1$. So its eigenvalues b_j lie in $[0, 1]$, and $\log\det(I + B_t) = \sum_j \log(1 + b_j) \geq \sum_j b_j \log 2 \geq \frac{1}{2}\operatorname{tr} B_t$. Since $\det V_{t+1} = \det V_t \det(I + B_t)$, summing gives $\sum_t \operatorname{tr} B_t \leq 2\log(\det V_{T+1}/\det V_1)$. Finally, $\operatorname{tr} V_{T+1} \leq d\lambda + KT$ and AM–GM give $\det V_{T+1} \leq (\lambda + KT/d)^d$. □

Proof of Theorem 2. Good event. Order items within a round arbitrarily. Features in a round are fixed before its rewards, so the sequence is predictable, and the self-normalized bound [1, Theorem 2], applied to $\theta^\star - \theta_0$ with targets $y - x^\top\theta_0$, gives with probability at least $1 - \delta/4$ for all t: $\|\hat\theta_t - \theta^\star\|_{V_t} \leq \beta_T$. The sequence has at most KT terms. So $|x^\top\hat\theta_t - x^\top\theta^\star| \leq \beta_T\|x\|_{V_t^{-1}}$ for every x by Cauchy–Schwarz. Combined with the validity event (probability at least $1 - \delta/2$), and writing $\bar\rho_{t,m} = \min(1, \hat\rho_{t,m} + \varepsilon_{t,m})$ and $\bar\mu_{t,m}$ for the clipped upper confidence bound,

$$\rho_{t,m} \leq \bar\rho_{t,m} \leq \rho_{t,m} + 2\varepsilon_{t,m}, \qquad \mu_{t,m} \leq \bar\mu_{t,m} \leq \mu_{t,m} + b_{t,m}, \quad b_{t,m} := \min(1, 2\beta_T\|x_{t,m}\|_{V_t^{-1}}).$$

Clipping preserves the lower bound as $\mu_{t,m} \in [0, 1]$. In particular $w_t^\star \leq \bar w_t$ coordinate-wise.

Per-round regret. Let $S_t^\star$ be optimal for $w_t^\star$. By coordinate monotonicity, the oracle property, and the Lipschitz property,

$$\alpha F_t(S_t^\star; w_t^\star) \leq \alpha F_t(S_t^\star; \bar w_t) \leq F_t(S_t; \bar w_t) \leq F_t(S_t; w_t^\star) + L \sum_{m \in S_t} (\bar w_{t,m} - w_{t,m}^\star).$$

Moreover, $\bar w - w^\star = \bar\rho\bar\mu - \rho\mu = \rho(\bar\mu - \mu) + (\bar\rho - \rho)\bar\mu \leq \rho\, b + 2\varepsilon$.

Summation. $b_{t,m}$ is determined by the history before round t, while $\mathbb{E}[Z_{t,m} \mid \text{history}] = \rho_{t,m}$. So $(\rho_{t,m} - Z_{t,m})b_{t,m}$ is a martingale difference sequence bounded by 1 in absolute value, with at most KT terms, and Azuma–Hoeffding gives, with probability at least $1 - \delta/4$, $\sum \rho b \leq \sum Z b + \sqrt{2KT\log(4/\delta)}$. The items with $Z = 1$ are exactly the ones entering V. By Cauchy–Schwarz and Lemma 2,

$$\sum_t \sum_{m \in S_t} Z_{t,m} b_{t,m} \leq 2\beta_T \sqrt{KT \sum_t \sum_{m : Z = 1} \|x_{t,m}\|^2_{V_t^{-1}}} \leq 2\beta_T \sqrt{2dKT \log(1 + KT/(d\lambda))}.$$

A union bound over the three events finishes the proof. □

Proof of Proposition 7. $G = \lambda \sum_i L_i \sim \text{Gamma}(n, 1)$ and $\lambda/\hat\lambda = G/n$. Chernoff bounds for the Gamma distribution give $\Pr[G/n \geq 1 + \eta] \leq \exp(-n(\eta - \log(1 + \eta)))$ and $\Pr[G/n \leq 1 - \eta] \leq \exp(-n(-\eta - \log(1 - \eta)))$. For $\eta \in (0, 1]$, $\eta - \log(1 + \eta) \geq \eta^2/6$ and $-\eta - \log(1 - \eta) \geq \eta^2/2$. So $\Pr[|G/n - 1| \geq \eta] \leq 2\exp(-n\eta^2/6)$. Set $\eta = \sqrt{6\log(2/\delta)/n}$, which is at most $\frac{1}{2}$ by the assumption on n.

On the complement write $g = G/n \in [\frac{1}{2}, \frac{3}{2}]$ so $\hat\lambda = \lambda/g$. By the mean value theorem, for some σ between 1 and $1/g$, so $\sigma \geq 2/3$, we have $|e^{-\lambda a/g} - e^{-\lambda a}| = |1/g - 1|\, \lambda a\, e^{-\sigma\lambda a}$. Since $\sup_{u \geq 0} u e^{-\sigma u} = 1/(e\sigma) \leq 3/(2e)$ and $|1/g - 1| = |1 - g|/g \leq 2\eta$, the difference is at most $3\eta/e$, for all a at once. □

Proof of Proposition 8. Let each round have m items with equal features, $\mu = 1$, $c = 1$, $B = 1$ and $F(S; w) = \sum_{i \in S} w_i$, with the index of the valid item uniform and independent across rounds. The information a blind learner gets from past rounds is independent of the current valid index, so whichever item it picks is valid with probability $1/m$ and $\mathbb{E}F_t(S_t) = 1/m$, while $\text{OPT}_t = 1$. □

Proof of Proposition 9. $\int_0^\infty e^{-\lambda a} \frac{\beta^\alpha \lambda^{\alpha-1} e^{-\beta\lambda}}{\Gamma(\alpha)}\, d\lambda = (\beta/(\beta + a))^\alpha$. The logarithm is $-\alpha\log(1 + a/\beta) = -\alpha\log a + \alpha\log\beta + o(1)$. A single access at age a has ACT-R activation $B = \log a^{-d} = -d\log a$. □

Proof of Theorem 3. Φ is monotone submodular (facility location), $\Phi(\emptyset) = 0$ and $\Phi(X) = 1$. For the greedy sequence with density-maximal g_i, as in the proof of Theorem 1, $\Phi(Z^\star) - \Phi(Z_{i-1}) \leq R^\star\Delta_i/c(g_i)$, so $d_i := \Phi(Z^\star) - \Phi(Z_i) \leq (1 - c(g_i)/R^\star)d_{i-1} \leq e^{-c(g_i)/R^\star} d_{i-1}$ whenever $d_{i-1} > 0$, using $1 - x \leq e^{-x}$ for all real x. If $d_{i-1} \leq 0$ for some i, then $\Phi(Z_\ell) \geq \Phi(Z^\star)$ by monotonicity. In every case $\Phi(Z_\ell) \geq (1 - e^{-R_\ell/R^\star})\Phi(Z^\star)$. Substituting $\Phi = 1 - D$ gives the first claim.

For the second, unit vectors satisfy $\|e_x - e_z\|^2 = 2(1 - \langle e_x, e_z\rangle)$. So $|\langle e_q, e_x - e_z\rangle| \leq \sqrt{2(1 - s(x, z))}$, which bounds the score change by $\ell_u\sqrt{2(1 - s(x, z(x)))}$. Average over p and apply Jensen's inequality to the square root, noting that $\mathbb{E}_p[1 - s(x, z(x))] = D(Z_\ell)$. □

Proof of Theorem 4. Condition on r and let T be the time until the first gap of length τ in a Poisson process of rate r started at the arrival. Let $A \sim \mathrm{Exp}(r)$ be the first access. If $A > \tau$ then $T = \tau$, otherwise $T = A + T'$ with T' an independent copy of T by the memoryless property. So $\mathbb{E}T = \mathbb{E}\min(A, \tau) + (1 - e^{-r\tau})\mathbb{E}T$. With $\mathbb{E}\min(A, \tau) = (1 - e^{-r\tau})/r$, this gives $\mathbb{E}T = (e^{r\tau} - 1)/r$.

The pairs (arrival time, lifetime) form a marked Poisson process with i.i.d. marks. By the marking theorem, the number of items alive at t is Poisson with mean $a \int_0^t \Pr[L > u]\, du$, which converges to $a\,\mathbb{E}L$ under the finiteness assumption. □